



Transportation's Systems 20-5751

PROJECT PART 0: PREREQUISITES AND BACKGROUND

PROF. ZAHRA AMINI
KEIVAN JAMALI

1405/01/23

1 Sioux Falls Transportation Network

The Sioux Falls network is one of the most widely used benchmark networks in transportation systems engineering. It was originally introduced in the 1970s during the U.S. Federal Highway Administration (FHWA) Urban Transportation Planning studies. Since then, it has become a standard dataset for evaluating traffic assignment algorithms, network optimization models, and computational methods in transportation research.

The network is a medium-scale synthetic representation of the road system of Sioux Falls, South Dakota. Although simplified, it captures a realistic structure and includes key characteristics such as:

- 24 nodes (intersections)
- 76 directed links (roadway segments)
- Multiple origin–destination (OD) pairs with associated travel demands
- Nonlinear link performance functions (often based on BPR)

Historically, this network gained popularity because it is complex enough to test real-world modeling challenges but small enough for students to implement algorithms efficiently. It is extensively used in:

- Traffic assignment experiments (user equilibrium, system optimal)
- Sensitivity analysis and calibration
- Machine learning and optimization studies
- Benchmarking new algorithms in network flow theory

Beyond academics, the Sioux Falls network has become a cultural element in the transportation research community—often regarded as the “MNIST of traffic assignment” due to its iconic status as an accessible yet meaningful dataset.

2 Required Python Libraries

Throughout this course and in the project parts that follow, students will implement and analyze transportation systems models using Python. The following libraries constitute the essential computational toolkit. Each library description includes a short explanation of its purpose and commonly used classes, modules, or functions that students will likely rely on.

2.1 NumPy

NumPy is the foundational package for numerical computing in Python. It provides efficient multi-dimensional arrays and linear algebra routines, making it essential for implementing mathematical models.

Key features:

- Fast array and matrix operations
- Vectorization to avoid slow Python loops
- Linear algebra tools (eigenvalues, matrix decomposition)

Commonly used components:

- `numpy.array`, `numpy.ndarray`
- `numpy.linalg` for matrix-based computations
- `numpy.random` for simulation

2.2 Pandas

Pandas provides high-level tools for data manipulation. It is invaluable when working with OD matrices, demand tables, link property files, or any tabular input data.

Key features:

- Easy data cleaning and filtering
- Import/export of CSV, Excel, and other file types
- Built-in statistics and grouping tools

Main components:

- `DataFrame` for tabular data
- `Series` for 1D labeled data
- `groupby` for aggregation

2.3 NetworkX

NetworkX is the most important library for this project. It provides a flexible framework for creating, modifying, and analyzing networks such as the Sioux Falls road network.

Key features:

- Graph construction (`Graph`, `DiGraph`)
- Shortest path algorithms (e.g., Dijkstra, Bellman-Ford)
- Network measures (centrality, connectivity)

Useful structures and functions:

- `networkx.DiGraph` for directed networks
- `G.add_node()`, `G.add_edge()`
- `nx.shortest_path()`, `nx.single_source_dijkstra()`

2.4 Matplotlib

Matplotlib is the standard library for plotting, useful for visualizing convergence, link performance functions, or network layouts.

Capabilities:

- Line plots (e.g., cost vs. flow)
- Scatter plots
- Network structure plots combined with NetworkX

Common modules:

- `matplotlib.pyplot`

2.5 SymPy

SymPy is a symbolic mathematics library. It helps with analytical derivations that appear in transportation modeling.

Key uses:

- Deriving gradients or Jacobians symbolically
- Working with BPR-type functions symbolically
- Checking equilibrium conditions analytically

Important tools:

- `symbols`, `Eq`, `solve`
- `diff` for derivatives

2.6 SciPy

SciPy extends NumPy with advanced algorithms for optimization, interpolation, and scientific computing. It becomes important when numerical solvers are required.

Relevant modules:

- `scipy.optimize` for solving equilibrium conditions
- `scipy.sparse` for large sparse matrices

2.7 Scikit-learn

Scikit-learn provides machine learning utilities. While not central to classical traffic assignment, it is helpful for:

- Clustering OD data
- Regression-based estimation of link parameters
- Classification of congestion patterns

Commonly used components:

- `LinearRegression`, `KMeans`
- `train_test_split`

2.8 TQDM

TQDM provides progress bars for loops, especially useful when implementing iterative traffic assignment algorithms such as FrankWolfe or Method of Successive Averages.

Key functionality:

- `tqdm(range(N))` to monitor iteration progress

2.9 Math and Statistics

The built-in `math` and `statistics` modules support essential computations without needing large packages.

Useful elements from `math`:

- `sqrt`, `log`, `exp`

Useful elements from `statistics`:

- `mean`, `median`, `variance`

3 Task

All subsequent parts of the project rely on a correctly constructed network model of Sioux Falls. In this section, students are required to build the directed graph using two provided CSV files. Completing this step ensures that the remaining components of the project can be performed consistently and without ambiguity.

3.1 Input Data Files

Two CSV files describe the full structure of the network:

- **SiouxFallsXY.csv** Contains the spatial coordinates of each node. Columns:
 - **node** Node identifier
 - **x** X coordinate (arbitrary planar system)
 - **y** Y coordinate
- **SoiuxFallsNetwork.csv** Contains the link-level connectivity and attributes. Columns:
 - **from** Starting node of the directed link
 - **to** Ending node of the directed link
 - **length** Physical length of the roadway segment(in meter)

These files together provide all the information needed to reconstruct the topology and geometry of the Sioux Falls network.

3.2 Required Task: Build the Network in Python

Students must write Python code to:

1. Load both CSV files using **pandas**.
2. Create an empty **networkx.DiGraph**.
3. Add nodes along with their spatial coordinates as node attributes.
4. Add directed edges according to the **from** and **to** fields of the link file, including **length** as an edge attribute.
5. (Optional but recommended) Generate a simple visualization of the network using **matplotlib** and the coordinate data.

The final constructed graph should satisfy:

- Exactly 24 nodes.
- Exactly 76 directed edges.
- Each node contains its associated **x** and **y** coordinates.
- Each link contains its **length** attribute.

This graph will be reused throughout subsequent project parts for tasks such as computing shortest paths, evaluating link performance, and performing traffic assignment.

3.3 Output Requirement

No submission on this part is required.